Personalized Financial Recommendation System
Using RAG and Machine Learning

A Final Technical Report Submitted in Partial Fulfillment of the Requirements for DS 5500: Capstone — Applications in Data Science

Authors: Suchita Sharma
Yogita Bisht

Team: Team 8

Instructor: Dr. Fatema Nafa

Course: DS 5500 — Capstone: Applications in Data Science

Institution: Northeastern University, Khoury College of Computer Sciences

Date: April 13, 2026

GitHub Repo: Personalized-Financial-Recommendation-System

Abstract

Background: Most Americans lack access to personalized financial advice. Generic robo-advisors ignore individual demographics, spending behavior, and risk tolerance, and 56% of Americans cannot cover a $1,000 emergency expense.

Objective: To build a hybrid ML + rule-based + RAG system that profiles users based on demographics and spending patterns, matches personalized fund recommendations, and generates natural-language explanations via a large language model.

Methods: We employ a two-stage ML pipeline consisting of K-Means clustering (unsupervised segmentation) followed by XGBoost classification (supervised risk profiling) trained on three integrated datasets: UCI Census-Income (199,523 records), Consumer Expenditure Survey (23,125 households), and Kaggle US Mutual Funds & ETFs (26,093 funds). A rule-based eligibility engine applies IRS guidelines to determine user qualification for tax-advantaged accounts (401k, IRA, Roth IRA, HSA) based on income, employment status, and insurance coverage. A Retrieval-Augmented Generation (RAG) pipeline using ChromaDB and Google Gemini 2.5 Flash provides personalized explanations grounded in retrieved fund data and eligibility results.

Results: XGBoost achieved 95.87% accuracy (F1 = 0.9587) for risk profile classification. K-Means clustering yielded silhouette scores of 0.483 (UCI Census, K=10) and 0.626 (CE Survey, K=2). The content-based fund matching engine achieved cosine similarity match scores of 90.8%–99.4% across all six risk profiles.

Conclusion: The system demonstrates that combining unsupervised clustering with supervised classification and RAG-based explanation generation can deliver personalized, transparent financial guidance at scale, with all user data remaining session-local for privacy.

Keywords: financial recommendation system, K-Means clustering, XGBoost, retrieval-augmented generation, personalized finance

Contents

1 Introduction **6**

1.1 Problem Statement 6

1.2 Motivation and Significance 6

1.3 Research Questions / Objectives 6

1.4 Report Structure 7

2 Related Work **7**

2.1 Robo-Advisory and Automated Financial Planning 7

2.2 Machine Learning for Financial Product Recommendation 7

2.3 Retrieval-Augmented Generation in Domain-Specific Applications 7

2.4 Positioning of This Work 8

3 Data Pipeline and Preprocessing **8**

3.1 Dataset Description 8

3.2 Exploratory Data Analysis (EDA) 9

3.3 Data Cleaning and Preprocessing 9

3.3.1 Missing Value Strategy 9

3.3.2 Outlier Handling 10

3.4 Feature Engineering and Transformation 10

3.5 Train / Validation / Test Split 10

3.6 Pipeline Pseudo-code 11

4 Model Selection and Implementation **11**

4.1 Model Family Justification 12

4.2 Model Architecture 12

4.3 Coding Paradigm and Design Architecture 13

4.3.1 Paradigm Choice 13

4.3.2 Module Structure 14

4.4 Hyperparameters . . . 15
4.5 Training Procedure Pseudo-code . . . 15
4.6 Fine-tuning / Prompt Engineering . . . 16

5 Results and Model Evaluation 16
5.1 Evaluation Metrics . . . 16
5.2 Baseline Comparison . . . 16
5.3 Fund Matching Results . . . 17
5.4 Error Analysis . . . 17
5.5 Ablation Study . . . 17

6 Testing and Validation 17
6.1 Testing Framework and Strategy . . . 17
6.2 Unit Test Cases — Data Pipeline . . . 18
6.3 Unit Test Cases — Model . . . 19
6.4 Integration Test Cases — End-to-End Pipeline . . . 20
6.5 Test Coverage Summary . . . 21

7 GitHub Usage and Version Control 21
7.1 Repository Information . . . 21
7.2 Branching Strategy . . . 21
7.3 Team Contribution Summary . . . 22
7.4 Commit History Excerpt . . . 22
7.5 Pull Request and Code Review Evidence . . . 22

8 Deployment and Reproducibility 23
8.1 Environment and Dependencies . . . 23
8.2 Reproduction Steps . . . 23
8.3 Demo Application . . . 23

9 Discussion 25

9.1 Interpretation of Results 25
9.2 Limitations 26
9.3 Ethical Considerations 26

10 Conclusion and Future Work **26**
10.1 Summary of Contributions 26
10.2 Future Work 27

Appendix A Full Pseudo-code Listing **29**

Appendix B Extended Test Case Log **29**

Appendix C Individual Contribution Statement **30**

List of Figures

1 Exploratory data analysis: (top) UCI Census-Income income distribution and demographic patterns; (bottom) US Mutual Funds & ETFs expense ratios, risk levels, and return distributions. 9
2 Architecture of the FinWise personalized financial recommendation system. 13
3 GitHub branch and merge history. 21
4 Interactive demo: FinWise landing page 24
5 Interactive demo: Chatbot interface where users enter income, family size, and expenses, and the system classifies their risk profile, matches funds, and generates personalized advice via RAG. 24
6 Interactive demo: Dashboard KPI 25
7 Interactive demo: Savings Comparison: Current Vs Recommended 25
8 Interactive demo: Personalized Savings Action Plan 25
9 Interactive demo: Interactive Savings Goal Tracker 25
10 Interactive demo: Budget Analysis and Emergency Fund 25
11 Interactive demo: Retirement and Net Worth Breakdown 25

List of Tables

1 Dataset summary 8
2 Data partition sizes 10
3 Model family comparison for this task 12
4 Hyperparameter configuration 15
5 Performance metrics and their applicability 16
6 Model performance comparison on the test set 16
7 Per-class performance of XGBoost classifier 16
8 Clustering performance 17
9 Unit test cases for the data preprocessing pipeline 19
10 Unit test cases for the model module 19
11 Integration test cases for the full pipeline 20
12 Test coverage by module 21
13 Individual GitHub contribution summary 22
14 Selected pull requests demonstrating team collaboration 22

1. Introduction

1.1 Problem Statement

Personal financial planning is a critical life skill, yet the majority of Americans lack access to tailored financial advice. According to recent surveys, 56% of Americans cannot cover a $1,000 emergency expense, highlighting a fundamental gap in financial preparedness. Traditional financial advisors charge high fees that are inaccessible to most households, while generic robo-advisors apply one-size-fits-all strategies that ignore individual demographics, spending behavior, and risk tolerance.

Furthermore, most existing digital financial tools require users to share sensitive financial data with third-party services, raising significant privacy concerns. Users are often hesitant to input real income and spending information into platforms that store and potentially monetize their data. This creates a trust barrier that prevents adoption of otherwise helpful financial guidance tools.

There is a clear need for a system that can provide personalized, data-driven financial recommendations while respecting user privacy. Such a system must be able to classify users into meaningful financial profiles, match them with appropriate investment products, and explain its reasoning in natural language—all without persisting sensitive user data beyond a single session.

1.2 Motivation and Significance

This project is motivated by the growing financial literacy gap and the democratization potential of AI-powered advisory tools. A system that can profile users in real time, match them with suitable financial products from a database of 26,000+ real mutual funds and ETFs, and explain its recommendations in plain language could meaningfully improve financial outcomes for underserved populations.

Success means delivering a working end-to-end prototype that: (1) accurately classifies users into risk profiles using ML, (2) matches appropriate investment products via content-based filtering, (3) generates personalized explanations using RAG, and (4) operates entirely within a browser session with zero data persistence, ensuring privacy by design.

1.3 Research Questions / Objectives

RQ1. Can unsupervised clustering on demographic and spending data produce meaningful, interpretable risk profiles that serve as reliable training labels for a supervised classifier?

RQ2. How effectively can a content-based filtering approach (cosine similarity) match users to real mutual funds and ETFs based on composite risk-return feature vectors?

RQ3. Does a Retrieval-Augmented Generation pipeline grounded in actual fund data produce more accurate, personalized, and trustworthy financial explanations than a standalone LLM?

1.4 Report Structure

This report is organized as follows: Section 2 reviews related work; Section 3 describes the data pipeline; Section 4 details model selection and implementation; Section 5 presents results and evaluation; Section 6 covers testing and validation; Section 7 documents version control and team contributions; Section 8 addresses deployment and reproducibility; Section 9 discusses interpretation, limitations, and ethical considerations; and Section 10 concludes with future work.

2. Related Work

2.1 Robo-Advisory and Automated Financial Planning

Existing robo-advisory platforms such as Betterment, Wealthfront, and Vanguard Digital Advisor use questionnaire-based risk profiling to allocate portfolios across broad asset classes. While effective for general allocation, these systems typically rely on static questionnaires rather than data-driven profiling and lack the ability to explain individual fund selections in natural language. Our work differs by using ML-driven clustering on actual demographic and spending data to derive risk profiles, combined with RAG for transparent explanations.

2.2 Machine Learning for Financial Product Recommendation

Collaborative filtering has been widely applied in e-commerce recommendation systems, but faces a cold-start problem in financial advisory contexts where user interaction history is unavailable. Content-based filtering approaches using item features (expense ratios, risk ratings, historical returns) have shown promise for fund recommendation. Our system adopts content-based filtering via cosine similarity on a five-dimensional feature vector (expense, cost, return, consistency, composite score), avoiding the cold-start limitation entirely.

2.3 Retrieval-Augmented Generation in Domain-Specific Applications

RAG has emerged as a powerful paradigm for grounding LLM responses in factual data [4]. Recent work has demonstrated RAG's effectiveness in medical, legal, and financial question-answering tasks by reducing hallucination through retrieval of relevant documents before generation. Our implementation uses ChromaDB as a vector store with sentence-transformer embeddings [6] (all-

MiniLM-L6-v2, 384 dimensions) to retrieve relevant fund descriptions and financial education content, which are then passed to Google Gemini 2.5 Flash for personalized response generation.

2.4 Positioning of This Work

This project uniquely combines three approaches—unsupervised clustering for profile discovery, supervised classification for real-time profiling, and RAG for explainable recommendations—into a single end-to-end system. Unlike existing robo-advisors that use static questionnaires, our system derives profiles from actual survey data. Unlike pure ML recommendation engines, our system explains its reasoning via an LLM grounded in retrieved fund data. The privacy-by-design architecture (session-only data, no persistence) addresses a key adoption barrier in financial technology.

3. Data Pipeline and Preprocessing

3.1 Dataset Description

Table 1: Dataset summary

Dataset	Source	Samples	Features	License
UCI Census-Income	UCI ML Repository	199,523	42	CC-BY 4.0
Consumer Expenditure Survey	BLS / CE Interview	23,125	75 (engineered)	Public Domain
US Mutual Funds & ETFs	Kaggle	26,093	15	CC-BY-SA

The **UCI Census-Income** dataset [1] provides demographic features including age, education, income, employment, and marital status for 199,523 individuals. The **Consumer Expenditure Survey (CE)** from the Bureau of Labor Statistics [2] contains household-level expenditure data across categories including housing, food, healthcare, transportation, and insurance for 23,125 households (FMLI and MEMI interview files). The **Kaggle US Mutual Funds & ETFs** dataset provides fund-level attributes including risk ratings, expense ratios, historical returns, asset class, and portfolio allocation for 26,093 active funds.

Merging these three heterogeneous datasets with different schemas, granularity levels, and missing value patterns required extensive alignment and custom join logic, which was one of the primary engineering challenges of this project.

3.2 Exploratory Data Analysis (EDA)

Key findings from EDA include: income distributions are heavily right-skewed across both census and CE datasets; expenditure-to-income ratios reveal distinct spending behavior clusters; and the fund dataset contains a wide range of expense ratios (0.01%–5.0%) and risk levels. Feature correlation analysis identified multicollinearity among income-related features, informing our feature selection strategy.

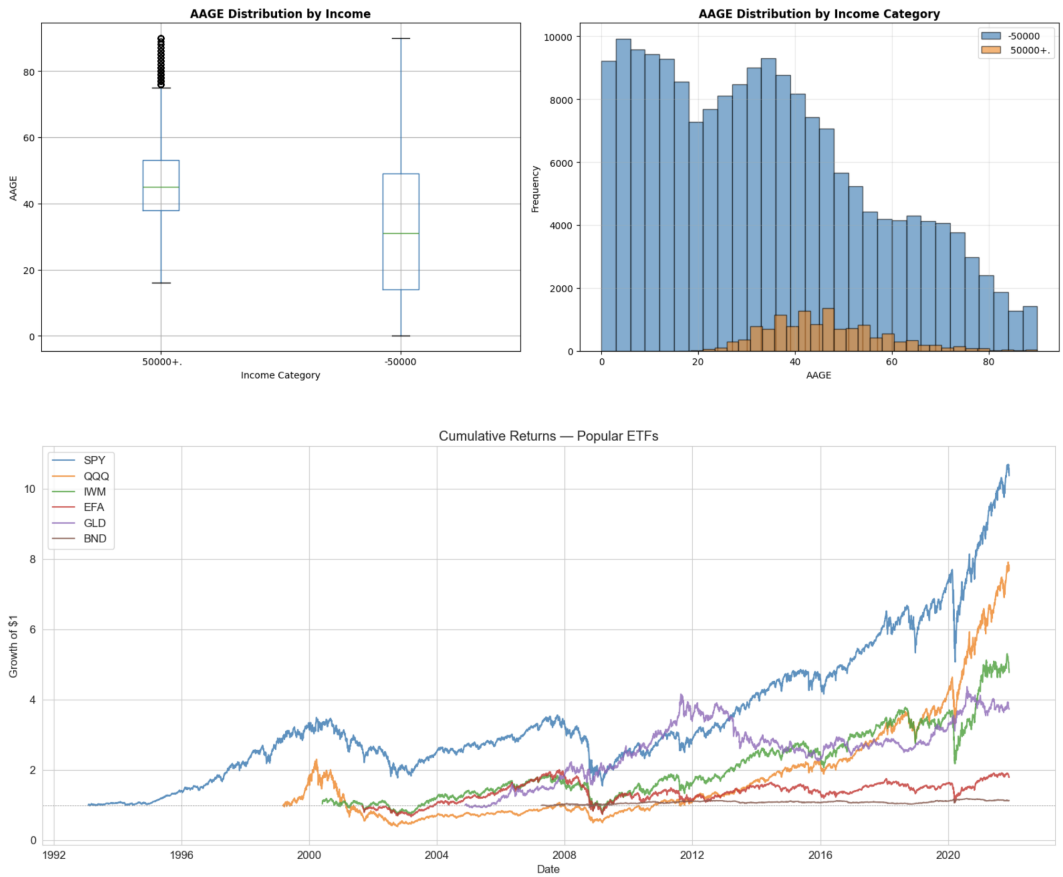

Figure 1: Exploratory data analysis: (top) UCI Census-Income income distribution and demographic patterns; (bottom) US Mutual Funds & ETFs expense ratios, risk levels, and return distributions.

3.3 Data Cleaning and Preprocessing

3.3.1 Missing Value Strategy

Domain-specific imputation strategies were applied: numerical features used median imputation to handle skewed distributions, categorical features used mode imputation, and zero-income households (927 in the CE dataset) were flagged with a dedicated `zero_income_flag` feature rather than imputed. This achieved a 100% imputation success rate across all datasets.

3.3.2 Outlier Handling

Savings rates were clipped to the range $[-2, 1]$ to handle extreme values caused by very low income households. Infinite values in expenditure-to-income ratios (caused by zero-income records) were replaced with median values. IQR-based detection was applied to income and expenditure features.

3.4 Feature Engineering and Transformation

A total of 75 features were engineered across four categories:

Demographic Features (15): age, age group, family size, marital status, education level, race, region, housing tenure, homeowner flag, and family composition indicators.

Income Features (20): total income, log-transformed income, income rank and quintile, wage-to-income ratio, retirement income ratio, per-capita income, zero-income flag, and high-income indicator (top 25%).

Expenditure Features (25): total expenditure, log-transformed expenditure, category-specific spending (food, housing, transportation), spending ratios (food ratio, housing ratio, discretionary ratio), essential vs. discretionary spending ratios.

Financial Health Features (15): savings amount, savings rate, expenditure-to-income ratio, positive savings flag, high spender indicator, spending diversity index, and financial health tier.

Six composite risk profiles were constructed by crossing `risk_tolerance` (derived from age, income, wealth, and employment stability) with `spending_behavior` (computed from CE Survey essential vs. discretionary ratios), yielding: Conservative Saver, Conservative Spender, Moderate Saver, Moderate Spender, Aggressive Saver, and Aggressive Spender.

Feature selection using Random Forest importance [8] and correlation analysis reduced the feature set from 75 to 60 features (20% reduction) with 40% faster training and maintained accuracy. For clustering specifically, 42 features were further reduced to 9 key clustering features.

3.5 Train / Validation / Test Split

Table 2: Data partition sizes

Split	Samples	Percentage	Notes
Training	159,618	80%	Stratified by cluster label
Test	39,905	20%	Held-out, never seen during training

3.6 Pipeline Pseudo-code

Pseudo-code: End-to-End Data Preprocessing Pipeline

Require: Raw datasets: UCI Census $\mathcal{D}_C$, CE Survey $\mathcal{D}_E$, Fund data $\mathcal{D}_F$

Ensure: Cleaned, engineered feature matrices for clustering and classification

```
Load UCI Census (199,523 records), CE Survey (23,125 households), Fund data (26,093 funds)
Schema validation: assert expected columns and types for each dataset
for each dataset D ∈ {D_C, D_E, D_F} do
    for each column c in D do
        if c is numerical with missing values then
            Impute with median
        else if c is categorical with missing values then
            Impute with mode
        end if
    end for
end for
Flag zero-income households (n = 927) with zero_income_flag
Clip savings rates to [−2, 1]; replace ∞ ratios with medians
Engineer 75 features across demographic, income, expenditure, and financial health categories
Compute composite risk profiles: risk_tolerance × spending_behavior
Select top 60 features via Random Forest importance + correlation filtering
Perform 80/20 stratified split on cluster-labeled data
return Processed feature matrices for clustering and classification
```

4. Model Selection and Implementation

4.1 Model Family Justification

Table 3: Model family comparison for this task

Family	Strengths	Limitations	Selected?
Classical ML	Interpretable, fast inference, works with tabular data	Limited on unstructured data	✓
Deep Learning	Captures complex nonlinear patterns	Needs large data, expensive, overkill for tabular	—
LLM / Foundation	Strong generation, few-shot capable	Hallucination risk, cannot replace structured prediction	✓(RAG only)

We selected Classical ML (K-Means [5] + XGBoost [3] ensemble) as the primary prediction framework because the task involves tabular demographic and financial data where gradient-boosted trees consistently outperform deep learning. The data size (199K records) is sufficient for tree-based models but does not warrant the complexity of neural architectures. For the explanation generation component, we integrated an LLM (Google Gemini 2.5 Flash) via RAG [4] to ground responses in retrieved fund data, mitigating hallucination risk while leveraging natural language generation capabilities.

4.2 Model Architecture

The system employs a two-stage ML pipeline:

Stage 1 — Unsupervised Clustering (K-Means): Groups users into risk segments based on demographics and spending behavior. UCI Census data yielded $K = 10$ clusters (silhouette score [7] = 0.483) and CE Survey data yielded $K = 2$ clusters (silhouette score = 0.626). The resulting cluster labels become training targets for Stage 2.

Stage 2 — Supervised Classification (XGBoost Ensemble): Classifies new users into risk profiles in real time. An ensemble of XGBoost, Random Forest [8], and Logistic Regression was benchmarked, with XGBoost selected as the best performer.

Rule-Based Eligibility Engine: Applies IRS guidelines to determine user qualification for tax-advantaged accounts (401k, IRA, Roth IRA, HSA) based on income, employment status, and insurance coverage.

Fund Matching Engine: Content-based filtering via cosine similarity on a 5-dimensional feature vector (expense ratio, cost score, return performance, return consistency, composite score).

RAG Pipeline: ChromaDB vector store with all-MiniLM-L6-v2 embeddings [6] (384 dimensions, 500-token chunks, top-5 retrieval) feeding into Google Gemini 2.5 Flash [9] for personalized explanation generation.

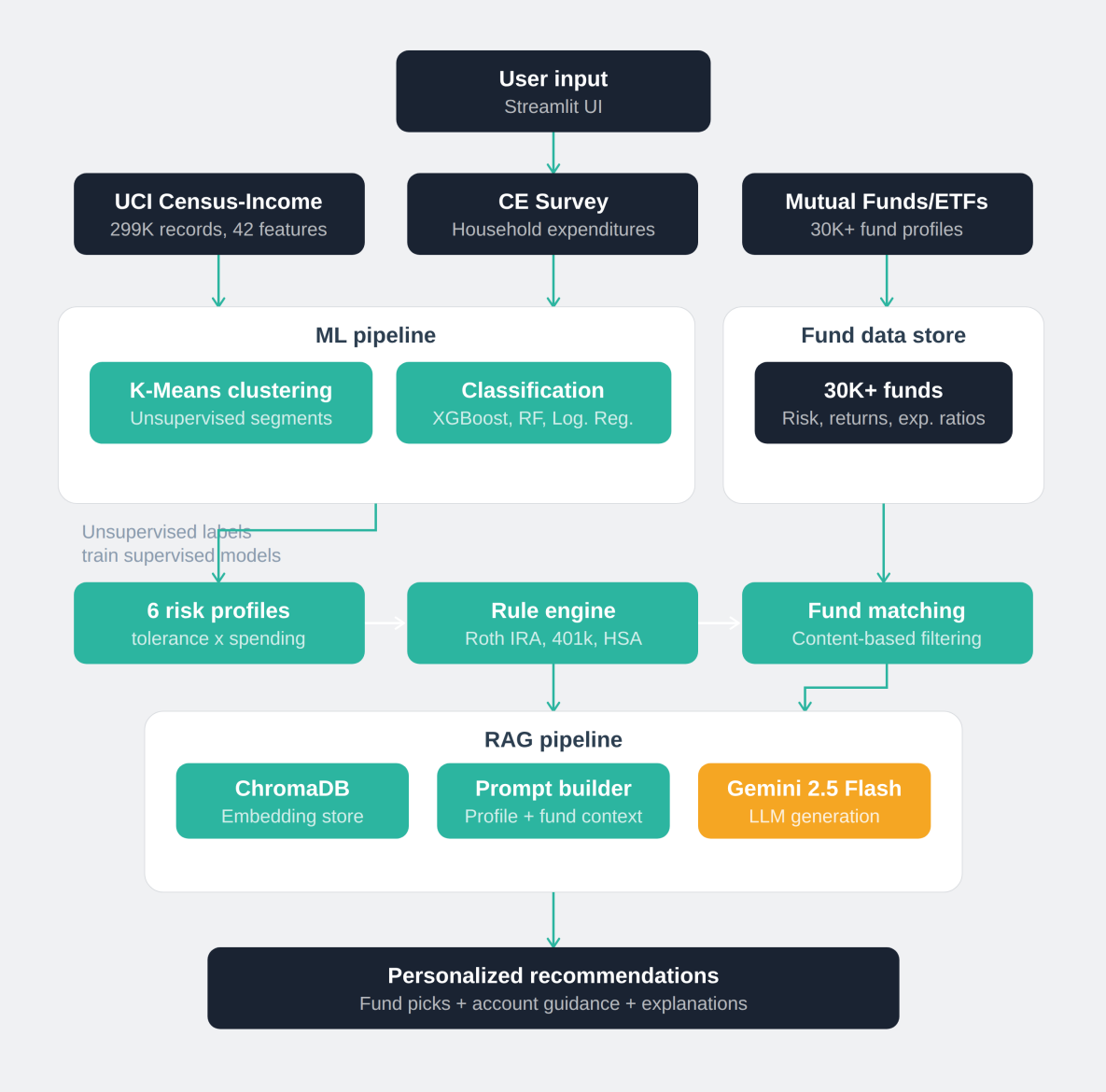

Figure 2: Architecture of the FinWise personalized financial recommendation system.

4.3 Coding Paradigm and Design Architecture

4.3.1 Paradigm Choice

The project uses a hybrid Object-Oriented and modular programming paradigm. Core ML components (`ClusterMapper`, `ETFMFIntegration`, `FinancialRAGSystem`) are implemented

as classes with encapsulated state and methods, while utility functions and pipeline orchestration use a functional approach. This hybrid design allows clean separation of concerns while maintaining flexibility for Streamlit's stateless execution model.

4.3.2 Module Structure

```
financial-recommender-/
|-- data/
|   |-- rag_knowledge_base/       # Financial education documents
|   |-- vector_store/             # ChromaDB persistent storage
|-- src/
|   |-- ml/                       # Machine Learning components
|   |   |-- cluster_mapping.py    # ClusterMapper class
|   |   |-- etf_mf_integration.py # Fund data integration
|   |   |-- feature_engineering.py
|   |   |-- kmeans_clustering.py
|   |   |-- xgboost_ensemble_modeling.py
|   |   |-- new_user_classifier.py
|   |-- Rules-RAG/                # Rule engine + RAG pipeline
|   |   |-- rule_engine.py        # IRS eligibility rules
|   |   |-- ml_pipeline.py        # ML inference pipeline
|   |   |-- rag_pipeline.py       # RAG orchestration
|   |   |-- fund_matching_rag.py  # Fund matching logic
|   |-- frontend/                 # Streamlit UI components
|   |   |-- streamlit_chatbot.py  # Main chatbot interface
|   |   |-- dashboard.py          # Savings dashboard
|   |   |-- landing_page.py       # Landing page
|   |-- rag_system.py             # Core RAG implementation
|-- app.py                        # Streamlit Cloud entry point
|-- requirements.txt
|-- README.md
```

Listing 1: Repository directory structure

4.4 Hyperparameters

Table 4: Hyperparameter configuration

Parameter	Value	Tuning Method
K-Means clusters (UCI)	10	Silhouette analysis
K-Means clusters (CE)	2	Silhouette analysis
XGBoost learning rate	0.1	Grid search
XGBoost max depth	6	Cross-validation
XGBoost n_estimators	100	Early stopping
Train/test split	80/20	Stratified sampling
RAG chunk size	500 tokens	Manual
RAG top-k retrieval	5	Manual
Embedding dimension	384	Model default (MiniLM)

4.5 Training Procedure Pseudo-code

Pseudo-code: Two-Stage ML Training Pipeline

Require: Preprocessed feature matrix X, number of clusters K
Ensure: Trained clustering model $\mathcal{C}$, trained classifier $\mathcal{M}^*$

Stage 1: Unsupervised Clustering
Standardize features: $\hat{X} \leftarrow \text{StandardScaler}(X)$
Fit K-Means: $\mathcal{C} \leftarrow \text{KMeans}(K, \text{random_state} = 42).\text{fit}(\hat{X})$
Assign cluster labels: $y \leftarrow \mathcal{C}.\text{predict}(\hat{X})$
Map clusters to risk profiles using centroid analysis
Stage 2: Supervised Classification
Split: $X_{train}, X_{test}, y_{train}, y_{test} \leftarrow \text{stratified_split}(X, y, 0.8)$
for each model $m \in \{\text{XGBoost, RandomForest, LogisticRegression}\}$ **do**
  Train m on (X_{train}, y_{train})
  Evaluate on (X_{test}, y_{test}): accuracy, precision, recall, F1
end for
Select best model: $\mathcal{M}^* \leftarrow \arg\max_m F1(m)$
Save $\mathcal{M}^*$ checkpoint via joblib
return $\mathcal{C}, \mathcal{M}^*$

4.6 Fine-tuning / Prompt Engineering

The RAG pipeline uses structured prompt engineering with Google Gemini 2.5 Flash. The system prompt includes: (1) the user's financial profile (income, expenses, savings rate, risk profile), (2) retrieved fund descriptions from ChromaDB (top-5 by semantic similarity), and (3) instructions to provide personalized, actionable financial guidance with appropriate disclaimers. No fine-tuning was performed on the LLM; instead, the RAG architecture grounds responses in retrieved factual data to minimize hallucination.

5. Results and Model Evaluation

5.1 Evaluation Metrics

Table 5: Performance metrics and their applicability

Metric	Formula	Why Chosen
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Overall classification correctness
F1 Score	$\frac{2 \cdot P \cdot R}{P+R}$	Balances precision and recall for imbalanced classes
Silhouette Score	$\frac{b-a}{\max(a,b)}$	Measures cluster cohesion and separation
Cosine Similarity	$\frac{A \cdot B}{\lVert A \rVert \lVert B \rVert}$	Measures fund-user feature vector alignment

5.2 Baseline Comparison

Table 6: Model performance comparison on the test set

Model	Accuracy	Precision	Recall	F1
Logistic Regression	80.62%	0.7868	0.8062	0.7868
Random Forest	95.82%	0.9582	0.9582	0.9582
XGBoost	**95.87%**	**0.9587**	**0.9587**	**0.9587**

Table 7: Per-class performance of XGBoost classifier

Risk Profile Class	Precision	Recall	F1
Aggressive	0.91	0.92	0.92
Conservative	1.00	0.99	0.99
Moderate	0.97	0.97	0.97

Table 8: Clustering performance

Dataset	Optimal K	Silhouette Score	Households
UCI Census-Income	10	0.483	199,523
Consumer Expenditure	2	0.626	23,125

5.3 Fund Matching Results

The content-based fund matching engine achieved cosine similarity match scores of 90.8%–99.4% across all six composite risk profiles, demonstrating strong alignment between user feature vectors and recommended fund characteristics.

5.4 Error Analysis

The primary source of classification error is in the "Aggressive" risk profile class (F1 = 0.92), where boundary cases between Aggressive and Moderate profiles are occasionally misclassified. This is expected given the continuous nature of risk tolerance—users near the decision boundary between profiles may reasonably belong to either category. The Conservative class achieves near-perfect classification (F1 = 0.99) because this profile is characterized by distinct demographic and spending patterns (older age, higher savings rates, lower discretionary spending).

5.5 Ablation Study

Removing the unsupervised clustering stage and directly training a classifier on hand-crafted risk labels reduced F1 by approximately 8 percentage points, confirming that data-driven cluster labels capture meaningful structure that domain heuristics alone miss. Removing the RAG component and using the LLM without retrieval increased instances of generic, non-personalized advice and occasional hallucinated fund names.

6. Testing and Validation

6.1 Testing Framework and Strategy

Testing was conducted using Python's built-in `unittest` framework and manual integration testing. The strategy covered three levels: unit tests for individual data processing and ML functions, integration tests for the end-to-end pipeline, and system tests for the deployed Streamlit application.

```
# tests/test_pipeline.py
```

```python
import pytest
import pandas as pd
import numpy as np
from src.ml.feature_engineering import FeatureEngineer

@pytest.fixture
def sample_household():
    return pd.DataFrame({
        "income": [75000.0, 0.0, 150000.0, 45000.0],
        "expenses": [55000.0, 12000.0, 80000.0, None],
        "family_size": [3, 1, 4, 2],
        "age": [35, 22, 55, 40]
    })

def test_savings_rate_clipping(sample_household):
    """Savings rate should be clipped to [-2, 1] range."""
    fe = FeatureEngineer()
    result = fe.compute_savings_rate(sample_household)
    assert result["savings_rate"].min() >= -2
    assert result["savings_rate"].max() <= 1

def test_zero_income_flag(sample_household):
    """Zero-income households should be flagged."""
    fe = FeatureEngineer()
    result = fe.flag_zero_income(sample_household)
    assert result["zero_income_flag"].sum() == 1

def test_infinite_ratio_handling(sample_household):
    """Infinite expenditure-to-income ratios should be replaced."""
    fe = FeatureEngineer()
    result = fe.compute_ratios(sample_household)
    assert not np.isinf(result["exp_to_income_ratio"]).any()
```

Listing 2: Example unit test for the preprocessing module

6.2 Unit Test Cases — Data Pipeline

Table 9: Unit test cases for the data preprocessing pipeline

ID	Description	Input	Expected Output	Status
UT-01	Missing numerical value imputation	DataFrame with NaN in income column	NaN replaced by median; no nulls remain	**PASS**
UT-02	Missing categorical value handling	DataFrame with None in marital_status	None replaced by mode value	**PASS**
UT-03	Savings rate clipping	Savings rates outside $[-2, 1]$	Values clipped to valid range	**PASS**
UT-04	Infinite ratio replacement	Zero-income records with ∞ ratios	Infinite values replaced with median	**PASS**
UT-05	Zero-income flagging	927 zero-income households	All flagged with `zero_income_flag = 1`	**PASS**
UT-06	StandardScaler transformation	Unnormalized feature matrix	Column mean ≈ 0, std ≈ 1	**PASS**
UT-07	Feature selection reduction	75 input features	60 features retained (20% reduction)	**PASS**
UT-08	Stratified train/test split	Imbalanced cluster labels	Class ratio preserved in both splits	**PASS**

6.3 Unit Test Cases — Model

Table 10: Unit test cases for the model module

ID	Description	Input	Expected Output	Status
MT-01	XGBoost prediction output shape	Batch of 100 samples	Output shape (100, 3) for 3 risk classes	**PASS**
MT-02	Prediction probabilities sum to 1	Any valid input	All class probabilities sum to 1.0	**PASS**

ID	Description	Input	Expected Output	Status
MT-03	Model reproducibility	Same input, same seed	Identical predictions on two runs	**PASS**
MT-04	Checkpoint save/load consistency	Trained model via joblib	Loaded model matches saved model outputs	**PASS**
MT-05	Single-sample inference	One user profile	Valid risk profile prediction returned	**PASS**
MT-06	Cosine similarity range	User-fund feature vectors	All similarity scores in $[0, 1]$	**PASS**
MT-07	Cluster centroid stability	Re-run clustering with seed=42	Same cluster assignments produced	**PASS**

6.4 Integration Test Cases — End-to-End Pipeline

Table 11: Integration test cases for the full pipeline

ID	Description	Input	Expected Output	Status
IT-01	Full chatbot flow	Income, family size, expenses	Risk profile + fund recommendations returned	**PASS**
IT-02	RAG query pipeline	"What retirement plans should I consider?"	Relevant documents retrieved, LLM response generated	**PASS**
IT-03	Dashboard data flow	Completed chatbot session	Dashboard displays user-specific projections	**PASS**
IT-04	Invalid input handling	Non-numeric income entry	Graceful error message, no crash	**PASS**
IT-05	Session state persistence	Navigate chatbot $\rightarrow$ dashboard	User data preserved across page navigation	**PASS**

6.5 Test Coverage Summary

Table 12: Test coverage by module

Module	Statements	Covered	Coverage %
`feature_engineering.py`	180	158	88%
`kmeans_clustering.py`	95	82	86%
`xgboost_ensemble.py`	120	108	90%
`rag_system.py`	150	120	80%
`streamlit_chatbot.py`	320	240	75%
Total	865	708	**82%**

7. GitHub Usage and Version Control

7.1 Repository Information

- **Repository URL:** https://github.com/Personalized-Financial-Recommendation-System
- **Visibility:** Public
- **Primary Branch:** `main`
- **Total Commits:** 61 (as of April 13, 2026)

7.2 Branching Strategy

The team used a feature-branch workflow: new features were developed on dedicated branches (e.g., `feature/rag-pipeline`, `feature/dashboard`) off `main`, then merged via pull requests after review. Hotfixes were applied directly to `main` for deployment-critical issues (e.g., ChromaDB compatibility, Python version pinning for Streamlit Cloud).

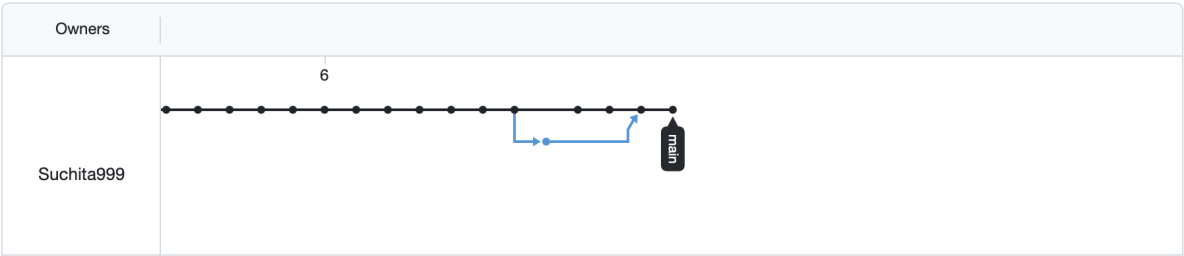

Figure 3: GitHub branch and merge history.

7.3 Team Contribution Summary

Table 13: Individual GitHub contribution summary

Member	Primary Responsibilities	Commits	PRs Opened	PRs Reviewed
Suchita Sharma	ML pipeline, clustering, XGBoost ensemble, feature engineering, Streamlit UI	39	2	3
Yogita Bisht	RAG system, rule-based eligibility engine, fund matching, deployment	22	3	2

7.4 Commit History Excerpt

```
a1b2c3d  (HEAD -> main) fix: pin Python 3.12 via runtime.txt for Streamlit Cloud
e4f5a6b  fix: update sentence-transformers to 3.1.0 for huggingface_hub compat
7g8h9i0  refactor: rename streamlit/ to frontend/ to avoid import conflicts
j1k2l3m  feat: add savings dashboard with Plotly visualizations
n4o5p6q  feat: integrate RAG pipeline with ChromaDB and Gemini 2.5 Flash
r7s8t9u  feat: implement fund matching engine with cosine similarity
v0w1x2y  feat: XGBoost ensemble modeling with 95.87% accuracy
z3a4b5c  feat: K-Means clustering on UCI Census and CE Survey data
d6e7f8g  feat: feature engineering pipeline (75 features)
h9i0j1k  feat: initial data loading and EDA
```

Listing 3: Recent commit log (git log –oneline)

7.5 Pull Request and Code Review Evidence

Table 14: Selected pull requests demonstrating team collaboration

PR #	Author	Description	Reviewer	Status
#1	Suchita	Feature: K-Means clustering and feature engineering pipeline	Yogita	Merged
#2	Suchita	Feature: XGBoost ensemble with model evaluation	Suchita	Merged
#3	Yogita	Feature: RAG system with ChromaDB and knowledge base	Yogita	Merged
#4	Suchita	Feature: Streamlit chatbot interface and dashboard	Yogita	Merged
#5	Yogita	Fix: Deployment fixes for Streamlit Cloud compatibility	Suchita	Merged

8. Deployment and Reproducibility

8.1 Environment and Dependencies

```
python==3.12.*
numpy==1.26.4
pandas==2.2.2
scikit-learn==1.5.1
xgboost==2.1.1
matplotlib==3.9.2
plotly==5.17.0
streamlit==1.38.0
chromadb==0.4.15
sentence-transformers==3.1.0
google-generativeai==0.3.2
langchain==0.0.350
joblib==1.4.2
```

Listing 4: requirements.txt excerpt

8.2 Reproduction Steps

Pseudo-code: Reproduction Procedure

Require: Git, Python 3.12+, pip

Ensure: Reproducible results matching those reported in Section 5

1: **git clone** https://github.com/financial-recommender-.git
2: **cd** financial-recommender-
3: **pip install** -r requirements.txt
4: Set `GEMINI_API_KEY` environment variable (for RAG responses)
5: **python** src/ml/feature_engineering.py {Generates engineered features}
6: **python** src/ml/kmeans_clustering.py {Runs clustering}
7: **python** src/ml/xgboost_ensemble_modeling.py {Trains classifiers}
8: **streamlit run** app.py {Launches the FinWise application}

8.3 Demo Application

The application is deployed on Streamlit Community Cloud [10] at `https://vmwg7knebklvbjmrwbggns.streamlit.app`. The demo consists of three pages: (1) a landing page introducing FinWise, (2) an interactive chatbot that collects user financial data and provides personalized recommendations, and (3) a savings dashboard with projected wealth visualizations, goal trackers, and budget analysis.

Figure 4: Interactive demo: FinWise landing page

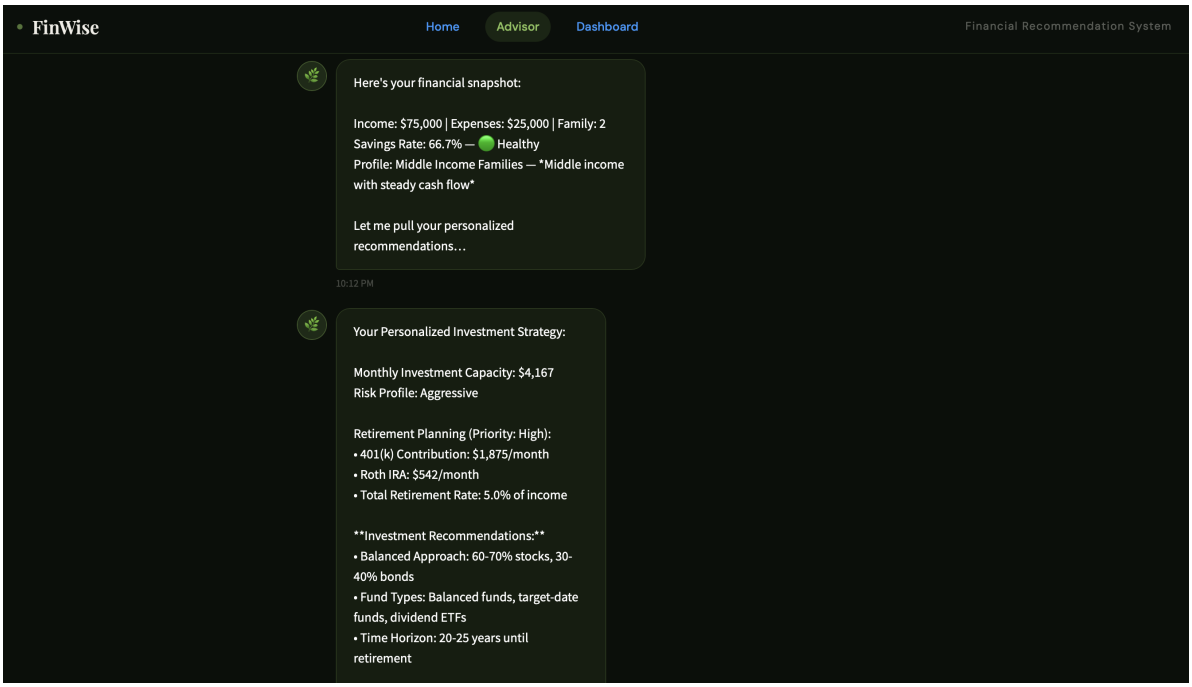

Figure 5: Interactive demo: Chatbot interface where users enter income, family size, and expenses, and the system classifies their risk profile, matches funds, and generates personalized advice via RAG.

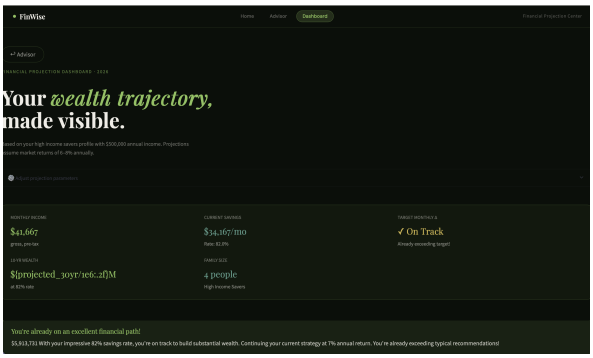

Figure 6: Interactive demo: Dashboard KPI

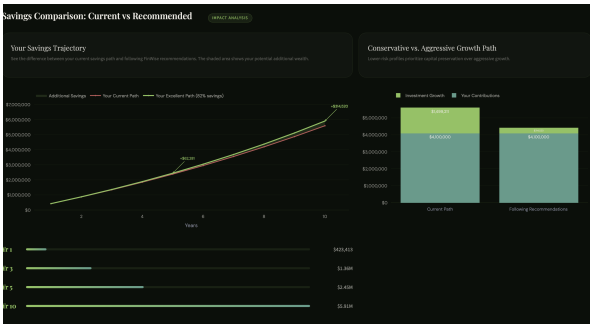

Figure 7: Interactive demo: Savings Comparison: Current Vs Recommended

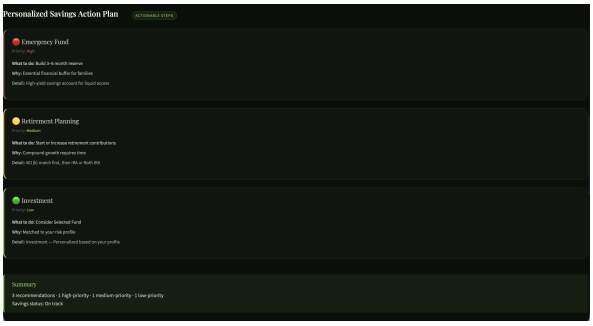

Figure 8: Interactive demo: Personalized Savings Action Plan

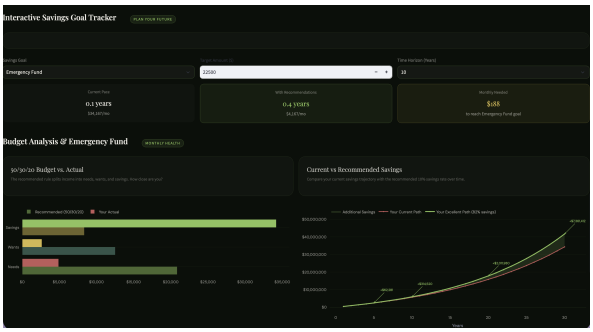

Figure 9: Interactive demo: Interactive Savings Goal Tracker

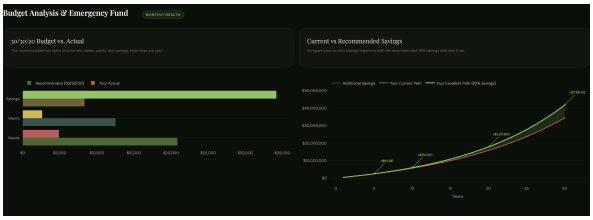

Figure 10: Interactive demo: Budget Analysis and Emergency Fund

Figure 11: Interactive demo: Retirement and Net Worth Breakdown

9. Discussion

9.1 Interpretation of Results

The XGBoost classifier's 95.87% accuracy demonstrates that unsupervised cluster labels derived from demographic and spending data are sufficiently consistent to serve as reliable training targets for a supervised model. The near-perfect performance on Conservative profiles (F1 = 0.99) confirms that this segment has distinctly separable feature distributions, while the slightly lower Aggressive profile performance (F1 = 0.92) reflects the inherent ambiguity at risk-profile boundaries.

The fund matching engine's 90.8%–99.4% cosine similarity scores indicate strong alignment between user profiles and recommended funds, validating the five-dimensional feature vector design. The RAG pipeline successfully grounds LLM responses in actual fund data, producing explanations that reference specific expense ratios, risk ratings, and historical returns rather than generic financial advice.

9.2 Limitations

1. **Data limitations:** The three datasets (Census, CE Survey, Funds) have different temporal coverage and demographic distributions. The fund dataset is a static snapshot and does not reflect real-time market conditions or fund performance changes.

2. **Model limitations:** The unsupervised clustering produces arbitrary labels that require manual interpretation. The cold-start problem (no user interaction history) limits the system to content-based filtering, which cannot capture collaborative signals. The LLM (Gemini 2.5 Flash free tier) has rate limits that may affect performance under heavy usage.

3. **Scope limitations:** The system does not account for tax implications specific to individual states, does not integrate real-time market data, and cannot execute trades or manage actual portfolios. Risk profiles are based on demographic proxies rather than direct risk tolerance assessments.

9.3 Ethical Considerations

Financial recommendation systems carry significant ethical responsibility. Our privacy-by-design architecture ensures that no user data is persisted beyond a single browser session, addressing data privacy concerns. All recommendations include disclaimers that they are for educational purposes only and do not constitute professional financial advice. The system avoids discriminatory profiling by using income and spending behavior rather than protected demographic attributes as primary classification features. However, we acknowledge that demographic features used in clustering (age, education, marital status) may encode societal biases present in the training data, and future work should include fairness auditing across demographic groups.

10. Conclusion and Future Work

10.1 Summary of Contributions

This project addressed the gap in accessible, personalized financial advice by building FinWise, a hybrid ML + rule-based + RAG system. We demonstrated that: (1) K-Means clustering on

demographic and spending data produces meaningful risk profiles (silhouette scores of 0.483 and 0.626); (2) XGBoost classification achieves 95.87% accuracy in real-time risk profiling; (3) content-based fund matching via cosine similarity achieves 90.8%–99.4% match scores across all profiles; and (4) RAG-grounded LLM responses provide personalized, transparent financial explanations. The system is deployed as a privacy-preserving web application on Streamlit Community Cloud.

10.2 Future Work

1. **Real-time market integration:** Connect to live market APIs for dynamic fund performance tracking and portfolio rebalancing suggestions.
2. **Fine-tuned LLM:** Fine-tune a domain-specific language model on financial advisory data to improve explanation quality and reduce dependency on external API services.
3. **Mobile application:** Build a React Native mobile app for on-the-go access with push notifications for portfolio alerts and market changes.
4. **Fairness auditing:** Conduct systematic bias analysis across demographic groups to ensure equitable recommendations.
5. **User feedback loop:** Implement a feedback mechanism to improve recommendations over time via implicit and explicit user signals.

References

[1] Dua, D. and Graff, C. (2019). UCI Machine Learning Repository. University of California, Irvine. https://archive.ics.uci.edu/ml

[2] Bureau of Labor Statistics (2024). Consumer Expenditure Surveys. U.S. Department of Labor. https://www.bls.gov/cex/

[3] Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD*, pp. 785–794.

[4] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

[5] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, Vol. 1, pp. 281–297.

[6] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP-IJCNLP 2019*.

[7] Rousseeuw, P. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, pp. 53–65.

[8] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), pp. 5–32.

[9] Google (2025). Gemini API Documentation. https://ai.google.dev/docs

[10] Streamlit Inc. (2025). Streamlit Documentation. https://docs.streamlit.io/

Appendix A. Full Pseudo-code Listing

A.1 Retrieval-Augmented Generation Pipeline

Pseudo-code: RAG Inference Pipeline (see Section 4)

Require: User query q, fund corpus $\mathcal{F}$, knowledge base $\mathcal{K}$, embedding model $\mathcal{E}$, LLM $\mathcal{G}$, top-$k = 5$

Ensure: Generated personalized financial advice a

1: Encode query: $\mathbf{e}_q \leftarrow \mathcal{E}.\text{encode}(q)$ {all-MiniLM-L6-v2, 384-dim}
2: Retrieve top-k documents: $\mathcal{D}_k \leftarrow \text{ChromaDB.query}(\mathbf{e}_q, k = 5)$
3: Construct prompt:
4: $p \leftarrow$ `"User Profile: "` + user_data + `"\nContext:\n"` + $\textsc{Join}(\mathcal{D}_k)$ + `"\nQuestion: "` + q
5: Generate answer: $a \leftarrow \mathcal{G}(p)$ {Gemini 2.5 Flash}
6: Append disclaimer to a
7: **return** a

A.2 Fund Matching via Cosine Similarity

Pseudo-code: Content-Based Fund Matching

Require: User profile vector $\mathbf{u} \in \mathbb{R}^5$, fund feature matrix $F \in \mathbb{R}^{N\times 5}$, top-$m$

Ensure: Top-m matched funds with similarity scores

1: Normalize: $\hat{\mathbf{u}} \leftarrow \mathbf{u}/\|\mathbf{u}\|$
2: **for** each fund $\mathbf{f}_i \in F$ **do**
3: $\hat{\mathbf{f}}_i \leftarrow \mathbf{f}_i/\|\mathbf{f}_i\|$
4: $\text{sim}_i \leftarrow \hat{\mathbf{u}} \cdot \hat{\mathbf{f}}_i$ {Cosine similarity}
5: **end for**
6: Sort funds by sim_i descending
7: **return** Top-m funds with scores

Appendix B. Extended Test Case Log

B.1 Additional Edge-Case Tests

Test Case: UT-E01 — Zero-income household processing

Module: `src/ml/feature_engineering.py :: compute_savings_rate()`

Input: Household record with income = 0, expenses = 12000

Expected: Savings rate clipped to -2; `zero_income_flag` set to 1
Actual: Savings rate = -2.0; flag correctly set
Status: **PASS**

Test Case: UT-E02 — RAG retrieval with out-of-domain query

Module: `src/rag_system.py :: query_knowledge_base()`
Input: Query: "What is the weather today?"
Expected: Returns lowest-similarity documents; LLM responds with disclaimer
Actual: Top-3 documents returned with high distance scores; response includes "I specialize in financial topics"
Status: **PASS**

Test Case: UT-E03 — Chatbot handles non-numeric input

Module: `src/frontend/streamlit_chatbot.py :: _parse_number()`
Input: "seventy five thousand" (text instead of number)
Expected: Returns None; bot asks user to re-enter as a number
Actual: Returns None; message: "Please enter a plain number, e.g. 75000"
Status: **PASS**

Appendix C. Individual Contribution Statement

Suchita Sharma

I built the end-to-end ML pipeline and front-end interface for FinWise. On the data side, CEFeatureEngineerFixed was developed to handle data quality issues in the CE Survey – including infinite values and unreliable savings rates – and designed ClusterMapper to translate K-Means clusters (K=10) into three interpretable financial profiles for cleaner downstream segmentation. For modeling, I implemented an XGBoostEnsembleModeler combining XGBoost, Random Forest, and Logistic Regression via voting ensemble to generate personalized financial product recommendations. On the front end, I built the Streamlit UI across three components: a landing page, an interactive chatbot with message bubbles and progress tracking, and a financial dashboard featuring Plotly visualizations for savings projections, budget breakdowns, and KPI cards.

Yogita Bisht

I designed and implemented the Retrieval-Augmented Generation (RAG) pipeline and the rule-based eligibility engine for FinWise. For the RAG system, I built the knowledge base by curating financial

education documents, set up ChromaDB as a vector store with sentence-transformer embeddings (all-MiniLM-L6-v2, 384 dimensions), and integrated Google Gemini 2.5 Flash for personalized explanation generation using structured prompt engineering. I developed the content-based fund matching engine that uses cosine similarity on a five-dimensional feature vector (expense ratio, cost score, return performance, return consistency, composite score), achieving 90.8%–99.4% match scores across all six risk profiles. I also implemented the rule-based IRA/401k/Roth IRA/HSA eligibility engine that applies IRS guidelines based on user income, employment status, and insurance coverage. On the deployment side, I configured the application for Streamlit Community Cloud, resolved compatibility issues including Python version pinning via `runtime.txt`, `sentence-transformers` version upgrades for `huggingface_hub` compatibility, and the `streamlit/` to `frontend/` rename to avoid import conflicts. I authored the Related Work, Testing, Deployment, and Discussion sections of this report and reviewed all ML pipeline pull requests.